PMPH Assignment 1

Bora Haliloglu

September 2026

Task 1: List Homomorphisms

Task 1.a

Assume that h is a well-defined list homomorphic function satisfying

$$\begin{aligned} h([]) &= e, \\ h([a]) &= f(a), \\ h(x + +y) &= h(x) \odot h(y), \end{aligned}$$

$++$ denotes list concatenation and $\odot$ is the binary operator associated with the list homomorphism.

We prove that

$$(\mathrm{Img}(h), \odot)$$

is a monoid with neutral element e. We need to prove associativity of $\odot$ and show that e is a left and right identity.

Associativity. Let

$$x, y, z \in \mathrm{Img}(h).$$

By the definition of the image of h, there exist lists a, b, c such that

$$x = h(a), \qquad y = h(b), \qquad z = h(c).$$

Then

$$\begin{aligned} (x \odot y) \odot z &= (h(a) \odot h(b)) \odot h(c) \\ &= h(a + +b) \odot h(c) \\ &= h((a + +b) + +c) \\ &= h(a + +(b + +c)) \\ &= h(a) \odot h(b + +c) \\ &= h(a) \odot (h(b) \odot h(c)) \\ &= x \odot (y \odot z). \end{aligned}$$

The fourth equality follows from associativity of list concatenation. Hence, $\odot$ is associative on $\mathrm{Img}(h)$.

Neutral element. First, note that

$$e = h([]),$$

so $e \in \mathrm{Img}(h)$.

Let $x \in \mathrm{Img}(h)$. Then there exists a list a s.t. $x = h(a)$. For the left identity we have

$$\begin{aligned} e \odot x &= h([]) \odot h(a) \\ &= h([] + +a) \\ &= h(a) \\ &= x. \end{aligned}$$

Similarly, for the right identity,

$$\begin{aligned} x \odot e &= h(a) \odot h([]) \\ &= h(a + +[]) \\ &= h(a) \\ &= x. \end{aligned}$$

Thus,

$$e \odot x = x \odot e = x \qquad \text{for all } x \in \mathrm{Img}(h).$$

Therefore, $\odot$ is associative and e is its neutral element. Hence,

$$\boxed{(\mathrm{Img}(h), \odot) \text{ is a monoid with neutral element } e.}$$

Task 1.b

We want to prove

$$(\mathrm{reduce}(+)\ 0) \circ (\mathrm{map}\ f) = (\mathrm{reduce}(+)\ 0) \circ \mathrm{map}\,((\mathrm{reduce}(+)\ 0) \circ (\mathrm{map}\ f)) \circ \mathrm{distr}_p\,.$$

$\circ$ denotes function composition.

We use the List Homomorphism Promotion Lemmas from lecture notes:

$$(\mathrm{map}\ f) \circ (\mathrm{map}\ g) = \mathrm{map}(f \circ g), \tag{1}$$

$$(\mathrm{map}\ f) \circ (\mathrm{reduce}(++)\ []) = (\mathrm{reduce}(++)\ []) \circ \mathrm{map}(\mathrm{map}\ f), \tag{2}$$

and

$$(\mathrm{reduce}(\odot)\ e) \circ (\mathrm{reduce}(++)\ []) = (\mathrm{reduce}(\odot)\ e) \circ \mathrm{map}(\mathrm{reduce}(\odot)\ e). \tag{3}$$

Furthermore, distr_p partitions a list into p sublists without changing its elements. Concatenating these sublists therefore reconstructs the original list:

$$(\mathrm{reduce}(++)\ []) \circ \mathrm{distr}_p = \mathrm{id}\,.$$

Starting from the left-hand side, we may therefore insert this identity:

$$\begin{aligned}&(\text{reduce}(+)\ 0) \circ (\text{map}\, f)\\ =\ &(\text{reduce}(+)\ 0) \circ (\text{map}\, f) \circ (\text{reduce}(++)\ []) \circ \text{distr}_p\,.\end{aligned}$$

We now apply Promotion Lemma (2) to

$$(\text{map}\, f) \circ (\text{reduce}(++)\ []).$$

This gives

$$= (\text{reduce}(+)\ 0) \circ (\text{reduce}(++)\ []) \circ \text{map}(\text{map}\, f) \circ \text{distr}_p\,.$$

Next, we apply Promotion Lemma (3), using $\odot = (+)$ and $e = 0$:

$$= (\text{reduce}(+)\ 0) \circ \text{map}(\text{reduce}(+)\ 0) \circ \text{map}(\text{map}\, f) \circ \text{distr}_p\,.$$

Finally, Promotion Lemma (1), i.e. map fusion, gives

$$\text{map}(\text{reduce}(+)\ 0) \circ \text{map}(\text{map}\, f) = \text{map}\,((\text{reduce}(+)\ 0) \circ (\text{map}\, f))\,.$$

Hence,

$$= (\text{reduce}(+)\ 0) \circ \text{map}\,((\text{reduce}(+)\ 0) \circ (\text{map}\, f)) \circ \text{distr}_p,$$

which is exactly the desired result. Therefore,

$$\boxed{(\text{reduce}(+)\ 0) \circ (\text{map}\, f) = (\text{reduce}(+)\ 0) \circ \text{map}\,((\text{reduce}(+)\ 0) \circ (\text{map}\, f)) \circ \text{distr}_p}.$$

Task 2: Longest Satisfying Segment

The parallel longest satisfying segment implementation uses the six element summary

$$(\mathit{lss}, \mathit{lis}, \mathit{lcs}, \mathit{len}, \mathit{first}, \mathit{last}).$$

The missing expressions in the reduction operator were as such:

```
let segments_connect = x_len == 0 || y_len == 0 || pred2 x_last y_first

let new_lss = max (max x_lss y_lss) (if segments_connect then x_lcs + y_lis else 0)

let new_lis = if x_lis == x_len && segments_connect then x_len + y_lis else x_lis

let new_lcs = if y_lcs == y_len && segments_connect then y_len + x_lcs else y_lcs

let new_len = x_len + y_len
```

The solution was validated using the `same`, `zeros`, and `sorted` predicates. The inputs produced the expected results 5, 5, and 9, respectively, and all tests passed.

The parallel and sequential implementations were benchmarked with the CUDA backend on random arrays of 10^6 elements. Speedup is computed as $T_{\text{seq}}/T_{\text{par}}$.

Test	Parallel (μs)	Sequential (μs)	Speedup
same	77	3,329,454	43,240×
zeros	75	3,428,331	45,711×
sorted	73	3,294,224	45,126×

On larger datasets, the parallel implementation achieves speedups of approximately 43,000–46,000× compared with the sequential implementation.

Task 3

Validation

The CUDA implementation was validated against the sequential implementation by comparing the output vectors. The validation condition was

$$|C_{\text{CPU}}[i] - C_{\text{GPU}}[i]| < 10^{-6}.$$

An epsilon of

$$\epsilon = 10^{-6}$$

was used. The implementation produced a `VALID` result for the small, medium, and large datasets.

CUDA Kernel and Launch Configuration

The CUDA kernel assigns one vector element to each GPU thread:

```
__global__ void vectorAddKernel(float *A, float *B,
                                float *C, unsigned int N)
{
    unsigned int gid = blockIdx.x * blockDim.x + threadIdx.x;

    if (gid < N) {
        C[gid] = A[gid] + B[gid];
    }
}
```

A block size of 256 threads was used:

```
#define BLOCK_SIZE 256
```

The number of blocks was computed as

```
unsigned int blocks = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
```

which corresponds to

$$N_{\text{blocks}} = \left\lceil \frac{N}{256} \right\rceil .$$

The kernel was launched as

```
vectorAddKernel<<<blocks, BLOCK_SIZE>>>(
    d_A, d_B, d_C, N);
```

The boundary check ensures correct execution when the vector length is not an exact multiple of the block size.

For GPU performance measurement, the kernel was executed 300 times. A `cudaDeviceSynchronize()` call was placed after the loop to ensure that all executions had completed before stopping the timer. The reported GPU runtime is the average runtime over these 300 executions. GPU memory allocation and host device memory transfers were excluded from the timing.

The sequential CPU implementation was executed and timed once.

Memory Throughput

Each vector addition performs two 4 byte reads and one 4 byte write:

$$A[i] \text{ (4 bytes)}, \qquad B[i] \text{ (4 bytes)}, \qquad C[i] \text{ (4 bytes)}.$$

Therefore, the total memory traffic per kernel execution is

$$12N \text{ bytes}.$$

The effective memory throughput was calculated as

$$\text{Throughput} = \frac{12N}{t_{\text{GPU}}},$$

where t_{GPU} is the average kernel runtime in seconds.

The measured results are shown in Table 1.

Dataset	Vector Length	Throughput (GB/s)
Small	20,057	47.945020
Medium	1,000,031	1150.195399
Large	100,000,009	1351.336255

Table 1: Measured CUDA memory throughput.

The small dataset achieves substantially lower throughput because it does not contain enough work to fully utilize the GPU. For such a short kernel execution, kernel launch and scheduling overhead also represent a larger fraction of the total execution time.

Throughput increases for the medium and large datasets because they provide sufficient parallel work to utilize the GPU effectively.

Medium and Large Dataset Performance

The medium dataset did not achieve higher throughput than the large dataset. The measured throughput was

1150.195399 GB/s

for the medium dataset and

1351.336255 GB/s

for the large dataset. The performance behavior mentioned in the task was not observed in these measurements.

The large dataset provides substantially more parallel work and better amortizes fixed kernel-launch and scheduling overhead, resulting in the highest measured throughput.

Sparse Matrix–Vector Multiplication

Implementation

The matrix is represented by:

- `mat_val`: pairs of the column index and value of each non zero matrix element,
- `mat_shp`: the # of stored elements in each matrix row,
- `vct`: the dense input vector.

The parallel implementation is shown below.

```
let spMatVctMult [num_elms][vct_len][num_rows]
                 (mat_val: [num_elms](i64, f32))
                 (mat_shp: [num_rows]i64)
                 (vct: [vct_len]f32)
                   : [num_rows]f32 =

  let shp_sc = scan (+) 0 mat_shp
  let starts_at = map2 (-) shp_sc mat_shp

  let flags = scatter (replicate num_elms false)
                      starts_at
                      (replicate num_rows true)

  let products = map (\(column, value) -> value * vct[column]) mat_val
```

```
  let sum = sgmSumF32 flags products

  in map (\row_end -> sum[row_end - 1]) shp_sc
```

The scan over `mat_shp` computes the ending position of each row in the flattened representation. Subtracting the row lengths gives the starting positions. These positions are used to construct the flags required by the segmented reduction.

The multiplication of every stored matrix value with the corresponding vector element is performed using `map`. The resulting products are then summed independently for each row using the segmented scan operation `sgmSumF32`. Finally, the last accumulated value of each segment is selected to form the output vector.

Experimental Setup

The sequential CPU version was compiled using

```
futhark c spMVmult-seq.fut
```

and the parallel GPU version using

```
futhark cuda spMVmult-flat.fut
```

A input dataset was generated for both implementations. The matrix contained 10,000 rows and 1,000,000 stored elements, corresponding to exactly 100 stored elements per row. The vector length was 10,000.

```
futhark dataset \
  --i64-bounds=0:9999 -g '[1000000]i64' \
  --f32-bounds=-7.0:7.0 -g '[1000000]f32' \
  --i64-bounds=100:100 -g '[10000]i64' \
  --f32-bounds=-10.0:10.0 -g '[10000]f32' \
  > large.in
```

Both implementations were executed 10 times on the same input:

```
./spMVmult-seq -t seq-times.txt -r 10 -n < large.in
./spMVmult-flat -t flat-times.txt -r 10 -n < large.in
```

The reported runtimes are the mean of the 10 measurements.

Performance Results

The measured execution times are shown in Table 2.

The acceleration was calculated as

$$S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}} = \frac{1.904}{0.145} \approx 13.12.$$

The CUDA-generated Futhark implementation achieved a speedup of

Implementation	Average Runtime (ms)
Sequential CPU	1.904
Parallel GPU	0.145

Table 2: Sparse matrix–vector multiplication runtimes.

$$\boxed{13.12\times}$$

relative to the sequential CPU implementation.

The result demonstrates that the flattened formula exposes data parallelism for efficient GPU execution.